



Model Development, Innovation, and Insights

COMPREHENSIVE REPORT

Introduction

A Telecom enterprise approached us with several challenges related to their core fraud detection platform. Their technology architecture was constrained by issues around data sovereignty, privacy, and residency. Due to data localization requirements, they had to deploy multiple Databricks instances across different geographies. This led to increased operational costs from both the spinning of data centers and the multiplication of Databricks license fees. Additionally, the system suffered from inefficient data science workflows. Our data science team identified these challenges and focus

Problem Statement

The project aimed to address the limitations of the Telecom Enterprise's previous fraud detection system, which used logistic regression models with pre-calculated features. Key issues included:

INFLEXIBILITY

The logistic regression model could not capture complex, non-linear relationships between features.

LIMITED SCOPE

The system was focused only on fraud detection and did not account for related tasks such as user classification or risk categorization.

FEATURE DEPENDENCY

The model was reliant on manually selected features, limiting its ability to extract complex patterns from raw data.

To overcome these limitations, we developed a multi-output deep learning model capable of predicting multiple aspects of fraud detection, including:

1. FRAUD SCORE PREDICTION

Estimating the likelihood of a transaction being fraudulent.

2. USER TYPE CLASSIFICATION

Classifying the user's risk level (e.g., high-risk or low-risk).

3. REASON CODE PREDICTION

Identifying the reasons a transaction was flagged or rejected.

This approach leverages deep learning to model complex, non-linear relationships and supports multiple tasks simultaneously.

Moreover, Feast, as a feature store, played a crucial role in the data management process. It ensured that we could:

CENTRALIZE FEATURE STORAGE

Feast provided a single source of truth for all features, simplifying their management and retrieval across multiple tasks.

REAL-TIME DATA SERVING

The ability to serve features in real-time was critical for fraud detection, where timely predictions are essential.

ENSURE FEATURE CONSISTENCY

Feast ensured that the same features were used during both training and production, maintaining model accuracy across environments.

INTEGRATION WITH MLFLOW

Feast's integration with MLFlow allowed us to track feature versions and monitor model performance, ensuring that changes in features did not negatively impact the model.

By carefully addressing data-related challenges, we were able to create a **high-quality, robust dataset** that supported the development of a multi-task deep learning model capable of solving fraud detection, risk user classification, and reason code generation more effectively than Telecom Enterprise's previous approach.

The main challenges faced during the data preparation and model training phase included:

- Handling missing values, class imbalance, outliers, and noisy data.
- Ensuring the dataset was suitable for multi-task learning.
- Managing temporal data and mitigating features engineering related problems.

The solutions we explored were:

- Applying data preprocessing techniques like imputation, balancing, normalization, and outlier removal.
- Leveraging **Feast** as a feature store to manage feature consistency and temporal changes.
- Ensuring data alignment and appropriate task-specific features for multi-task learning.

MODELS CREATED

We created a multi-output neural network model using TensorFlow/Keras, which consists of the following outputs:

FRAUD SCORE (REGRESSION)

A continuous output that predicts the fraud score of the transaction.

USER TYPE (CLASSIFICATION)

A categorical output that classifies the user type into one of several categories.

REASON CODES (CLASSIFICATION)

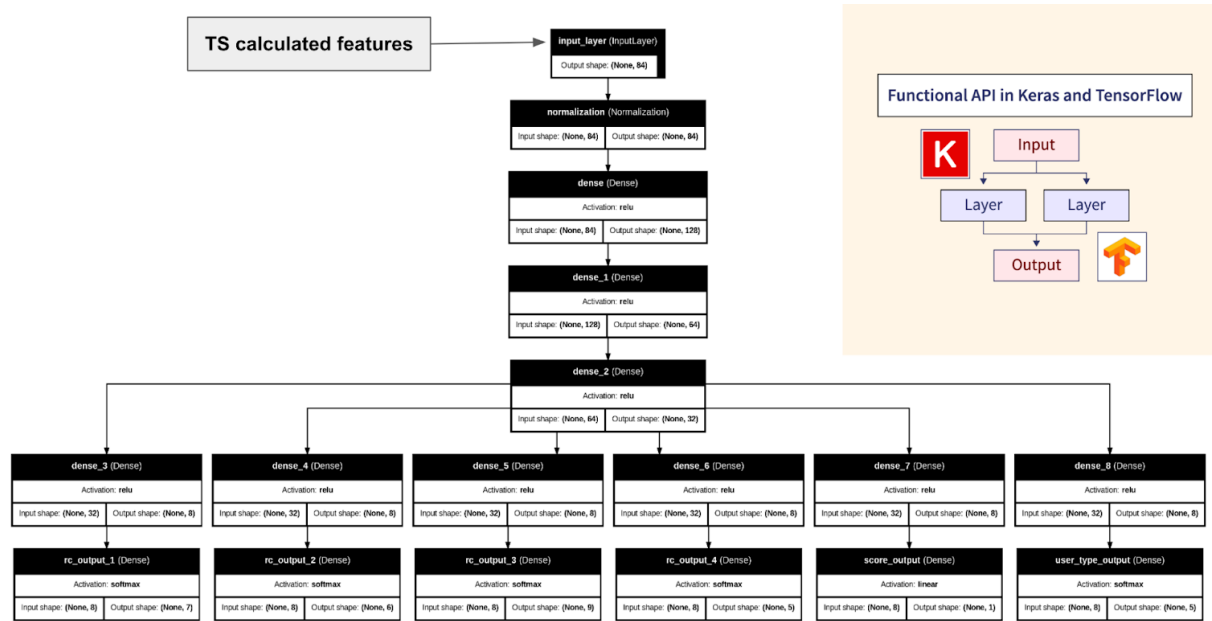
Four separate categorical outputs that predict the reason codes (e.g., different fraud categories or rejection reasons).

The model has multiple layers, with hidden layers designed to capture complex patterns in the data. The final layers are specific to each output type (e.g., softmax for classification and a custom activation function for regression).

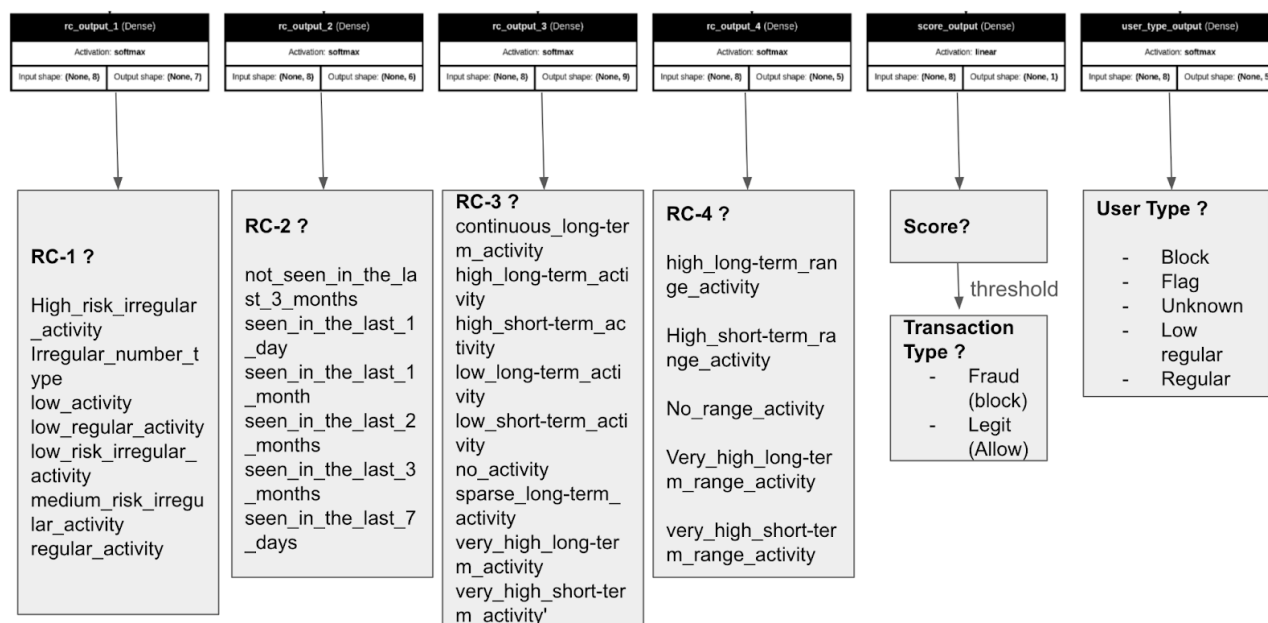
Why did we choose a particular model or algorithm to solve this problem?

CHOICE OF MODEL

The overall model architecture:



Different model outputs were defined as follow:



The decision to use a **multi-output deep neural network (DNN)** was driven by several key factors:

1. PROBLEM COMPLEXITY

The project required not only fraud detection but also user classification and reason code prediction. A DNN provides the flexibility to handle these diverse tasks within a shared architecture, which is ideal for **multi-task learning**. This approach enables the model to learn shared features across tasks, improving overall performance and eliminating the need for separate models.

2. LIMITATIONS OF PREVIOUS MODELS

Telecom Enterprise's old system relied on logistic regression, which struggled with the complexity of non-linear relationships in each task. Linear models are insufficient for modeling these interactions, limiting predictive accuracy. Additionally, each task (fraud detection, user classification, reason code prediction) was processed separately, leading to inconsistent predictions and high operational costs.

3. ADVANTAGES OF DEEP LEARNING

Non-linear Pattern Recognition:

DNNs excel at capturing non-linear relationships in data, which is crucial for fraud detection and user classification.

Shared Feature Learning:

A single model learns common features across tasks, enhancing efficiency and predictive accuracy.

Scalability:

The model can efficiently handle the large datasets typical in fraud detection systems.

4. TENSORFLOW/KERAS ECOSYSTEM

We chose **TensorFlow** and **Keras** for their flexibility, powerful GPU acceleration

5. EASE OF INTEGRATION

with other tools like MLflow and Feast, which are used for experiment tracking and feature management, respectively.

What was our approach to select the right algorithm?

APPROACH FOR ALGORITHM SELECTION

1. UNDERSTANDING AND DECOMPOSITION OF THE PROBLEM

The task required both regression (fraud score) and classification (user type, reason codes) predictions. We needed a model capable of handling these mixed-output tasks simultaneously, leading us to choose a multi-output deep learning model.

2. UNDERSTANDING THE FEATURE COMPLEXITY

The dataset contained complex features related to user and transaction information. A deep learning model was well-suited to capture the intricate relationships between these features.

3. EXISTING APPROACHES

Traditional models like logistic regression and random forests were considered but were found lacking in their ability to model non-linear relationships effectively, especially when multiple outputs were involved.

4. MODEL EXPERIMENTATION

We initially tested simpler models like decision trees, random forests, and logistic regression. However, these performed suboptimally, particularly in handling complex feature interactions. The multi-output DNN outperformed these models, especially in terms of predictive accuracy.

5. EVALUATION OF MODEL

We used TimeSeriesSplit cross-validation to ensure the model generalized well on unseen data. Key evaluation metrics included accuracy (for classification tasks) and mean squared error (MSE, for regression). MLFlow was utilized to systematically compare different model configurations, hyperparameters, and architectures.

6. DATA AND TASK CONSIDERATIONS

We evaluated the dataset, which included both regression (fraud score) and classification (user types and risk categories) tasks. The built multi-task model was optimal to handle these different types of tasks simultaneously.

7. MLFLOW FOR EXPERIMENT TRACKING

Throughout this process, we used MLFlow for experiment tracking, which allowed us to systematically compare different model configurations, hyperparameters, and architectures. This ensured that our approach was data-driven and transparent.

8. TRAINING WITH FEAST FEATURES STORE

Feast provided a centralized feature store, ensuring consistent access to high-quality, versioned features across training and production systems. This facilitated the management of feature drift and versioning challenges.

How do we train the model?

TRAINING PROCESS

1. DATA PREPROCESSING AND FEATURE MANAGEMENT

- **Feature Store:** We used Feast to store and manage features, ensuring consistency between training and production systems.
- **Preprocessing Steps:** Missing values, outliers, and class imbalance were addressed through imputation, normalization, and resampling.
- **Feature Engineering:** Features were preprocessed (e.g., one-hot encoding for categorical features), and normalization was applied to scale the data for neural network training.

3. LOSS FUNCTIONS

We use different loss functions for different outputs optimized simultaneously:

- **Categorical Cross-Entropy** for classification tasks (user type, reason codes).
- **Mean Squared Error (MSE)** for the regression task (fraud score).

5. VALIDATION

After training, the model is validated on the test set using the same performance metrics.

2. MODEL ARCHITECTURE

The neural network consists of several layers:

- **Input Layer:** Takes a vector of features.
- **Hidden Layers:** Multiple dense layers with ReLU activations to learn complex patterns.
- **Output Layers:** Separate output layers for each task:
- **Regression:** For predicting the fraud score (with a custom activation function to limit the maximum score value).
- **Classification:** For predicting user type and reason codes (with softmax activations).

4. TRAINING STRATEGY

- **Optimizer:** Adam optimizer with a learning rate of $1e-3$.
- **Techniques:** We applied early stopping, model checkpoints, and learning rate schedules to avoid overfitting.
- **Hyperparameter Tuning:** Grid search and random search were used to optimize hyperparameters such as learning rate, epochs, and batch size.

6. MODEL EVALUATION AND LOGGING

Post-training, the model was evaluated on a separate test set using metrics like accuracy, precision, recall, and F1-score for classification tasks, and MSE for regression tasks.

MLFlow: Used for experiment tracking, ensuring that all hyperparameters and performance metrics were logged for transparency and reproducibility.

What was the outcome?

MODEL PERFORMANCE

The final model achieved the following performance on the test set:

FRAUD SCORE PREDICTION (REGRESSION)

MSC	469.52
R ²	98.38%

USER TYPE CLASSIFICATION

Accuracy	95.55%
----------	--------

REASON CODE CLASSIFICATION

Accuracy RC_1 (Activity Type):	95.42%
Accuracy RC_2 (Seen in the last...):	38.17%
Accuracy RC_3 (Long/short-term activity)	70.66%
Accuracy RC_4 (Range activity)	98.97%

MODEL ROBUSTNESS

The model demonstrated consistent performance across multiple time splits during cross-validation, ensuring that it does not overfit or suffer from data leakage.

END-TO-END MODEL MANAGEMENT WITH FEAST AND MLFLOW

Feast: Ensured feature consistency between training and serving systems.

MLFlow: Facilitated comprehensive tracking of experiment results, improving the model development process.

SCALABILITY

The model is designed to handle large-scale datasets and can be deployed in real-time environments, making it suitable for large-scale fraud detection systems.

Why is it better than what exists today?

Why is our model superior?

ADVANTAGES OVER CLASSICAL EXISTING APPROACHES IN TELECOM ENTERPRISE

MULTI-OUTPUT PREDICTIONS

Unlike traditional models that predict a single output, our model can simultaneously predict multiple outputs, reducing the need for separate models and improving efficiency.

HANDLING COMPLEX RELATIONSHIPS

DNNs excel at capturing non-linear relationships, which is crucial for fraud detection where linear models often fail to capture user behavior and transaction patterns.

SCALABILITY

Our DNN can handle large datasets and remains robust as the data grows, unlike legacy systems that may struggle with scale.

INTEGRATION WITH FEAST

Using Feast ensures that features are up-to-date and that feature engineering pipelines are standardized, improving model reliability in production.

MODEL INTERPRETABILITY

While deep learning models can be more challenging to interpret, our approach allows us to track and analyze feature importance over time, providing insights into model behavior.

CROSS-VALIDATION & ROBUSTNESS

The use of TimeSeriesSplit cross-validation ensures that the model generalizes well and avoids overfitting, a challenge for traditional models working with sequential data.